

Prompt Manager Project

Python 콘솔 프로그램 개발 및 Git/GitHub 버전 관리 프로젝트

1. 프로젝트 개요

프로젝트명

Prompt Manager Project

프로젝트 목적

생성형 AI를 활용하면서 다양한 프롬프트를 작성하게 되었지만, 메모장, 노션, 메신저 등 여러 곳에 흩어져 있어 필요한 프롬프트를 빠르게 찾기 어려웠다.

이를 해결하기 위해 Python을 활용하여 프롬프트를 체계적으로 관리할 수 있는 콘솔 프로그램을 직접 구현하였다.

또한 단순히 프로그램을 작성하는 것에서 끝나는 것이 아니라 Git과 GitHub를 이용하여 프로젝트의 변경 이력을 관리하고 기능별 Commit, Branch 생성 및 Merge를 수행하여 실제 개발 프로젝트와 유사한 방식으로 진행하였다.

본 프로젝트를 통해 Python 기초 문법과 Git/GitHub 활용 방법을 함께 학습하는 것을 목표로 하였다.

프로젝트 목표

- Python 콘솔 프로그램 구현
- 리스트(List)와 딕셔너리(Dictionary)를 이용한 데이터 관리
- 함수(Function)를 활용한 기능 분리
- 반복문과 조건문을 이용한 프로그램 제어
- Git을 이용한 버전 관리
- GitHub 원격 저장소 관리
- Branch 생성 및 Merge 경험

- 기능 단위 Commit 작성

2. 개발 환경

구분	내용
운영체제	Windows
개발 도구	Visual Studio Code
프로그래밍 언어	Python 3.14.6
버전 관리	Git
원격 저장소	GitHub

개발 환경 구성

프로젝트는 Visual Studio Code를 이용하여 개발하였다.

Python 3.10 이상의 버전을 설치한 후 정상적으로 실행되는지 확인하였으며, Git을 설치하여 버전 관리 환경을 구성하였다.

GitHub 계정을 Visual Studio Code와 연동하여 원격 저장소를 사용할 수 있도록 설정하였다.

스크린샷 첨부

 01_개발환경_확인.png

```
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> python --version
Python 3.14.6
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git --version
git version 2.54.0.windows.1
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git config --global user
.name
김지민
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git config --global user
.email
jmkmo520@gmail.com
```

3. 프로젝트 구조

프로젝트는 아래와 같은 구조로 구성하였다.

```
Prompt_Manager_Project
|
```

```
|─ prompt_manager.py
|─ README.md
|─ .gitignore
└─ screenshots
```

파일 설명

파일명	설명
prompt_manager.py	프로그램 소스코드
README.md	프로젝트 설명 문서
.gitignore	Git에서 제외할 파일 설정
screenshots	프로젝트 진행 과정 캡처

프로젝트의 실행 파일은 **prompt_manager.py**이며, 프로젝트 설명은 **README.md**에 작성하였다.

또한 제출에 필요한 모든 실행 결과와 Git 작업 화면은 **screenshots** 폴더에 정리하였다.

4. Git 저장소 생성

프로젝트를 생성한 후 Git 저장소를 초기화하였다.

사용한 명령어는 다음과 같다.

```
git init
```

기본 브랜치를 **main**으로 변경하였다.

```
git branch -M main
```

저장소가 정상적으로 생성되었는지 확인하였다.

```
git status
```

GitHub 원격 저장소를 연결한 후 최초 Commit과 Push를 수행하였다.

사용한 명령어는 다음과 같다.

```
git add .
```

```
git commit-m"chore: initialize project structure"
```

```
git push
```

Git 저장소 생성 결과

Git 저장소가 정상적으로 생성되었으며, GitHub 원격 저장소와 연결하여 프로젝트 파일을 업로드하였다.

이후부터는 기능을 하나씩 구현할 때마다 Commit을 수행하여 변경 이력을 관리하였다.

 스크린샷 첨부

 02_git_init_main.png

```
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git init
Reinitialized existing Git repository in C:/Users/jmkmo/Desktop/Prompt_Manager_Project/.git/
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git branch -M main
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

 스크린샷 첨부

 04_first_commit.png

```

PS C:\Users\jmkim\Desktop\Prompt_Manager_Project> & 'C:\Users\jmkim\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\jmkim\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '50068' '--' 'c:\Users\jmkim\Desktop\Prompt_Manager_Project\prompt_manager.py'
Hello
PS C:\Users\jmkim\Desktop\Prompt_Manager_Project> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        prompt_manager.py

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\jmkim\Desktop\Prompt_Manager_Project> git add .
PS C:\Users\jmkim\Desktop\Prompt_Manager_Project> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   README.md
        new file:   prompt_manager.py

PS C:\Users\jmkim\Desktop\Prompt_Manager_Project> git commit -m "chore: initialize project structure"
[main (root-commit) 48bda80] chore: initialize project structure
3 files changed, 6 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 prompt_manager.py

```

 스크린샷 첨부

 05_first_push.png

```

PS C:\Users\jmkim\Desktop\Prompt_Manager_Project> git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 472 bytes | 78.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jmkim0520-glitch/Prompt_Manager_Project.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

```

5. 공개 샘플 저장소 Clone

기능 설명

과제 요구사항에 따라 GitHub의 공개 샘플 저장소를 `git clone` 명령어를 사용하여 로컬 컴퓨터로 내려받았다.

`git clone` 은 원격 저장소의 파일뿐만 아니라 커밋 이력과 브랜치 정보까지 함께 복사하는 Git 명령어이다.

샘플 저장소를 내려받은 후 저장소의 폴더 구조를 확인하고, Git 로그를 조회하여 정상적으로 저장소가 복사되었는지 확인하였다.

사용한 명령어

```
git clone https://github.com/octocat/Hello-World.git
```

```
cd Hello-World

dir

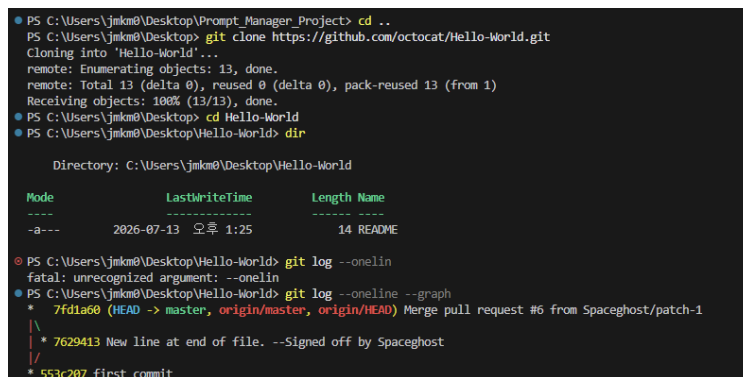
git log --oneline --graph
```

수행 결과

- 공개 GitHub 저장소를 로컬 컴퓨터로 성공적으로 복사하였다.
- 저장소의 폴더 구조를 확인하였다.
- `git log --oneline --graph` 명령어를 이용하여 커밋 이력을 확인하였다.
- clone 명령어의 사용 방법과 원격 저장소를 로컬 환경으로 가져오는 과정을 이해할 수 있었다.

스크린샷 첨부

25_git_clone_and_log.png



```
PS C:\Users\jnmk\Desktop\Prompt_Manager_Project> cd ..
PS C:\Users\jnmk\Desktop> git clone https://github.com/octocat/Hello-World.git
Cloning into 'Hello-World'...
remote: Enumerating objects: 13, done.
remote: Total 13 (delta 0), reused 0 (delta 0), pack-reused 13 (from 1)
Receiving objects: 100% (13/13), done.
PS C:\Users\jnmk\Desktop> cd Hello-World
PS C:\Users\jnmk\Desktop\Hello-World> dir

Directory: C:\Users\jnmk\Desktop\Hello-World

Mode                LastWriteTime         Length Name
----                -
-a---             2026-07-13 오후 1:25             14 README

PS C:\Users\jnmk\Desktop\Hello-World> git log --onelin
fatal: unrecognized argument: --onelin
PS C:\Users\jnmk\Desktop\Hello-World> git log --oneline --graph
* 7fd1a60 (HEAD -> master, origin/master, origin/HEAD) Merge pull request #6 from Spaceghost/patch-1
|
| * 7629413 New line at end of file. --Signed off by Spaceghost
|/
* 553c207 first commit
```

6. Git Pull 확인

수행 목적

과제 요구사항에 따라 `git pull` 명령어를 사용하여 원격 저장소의 최신 변경 사항을 로컬 저장소에 반영하는 과정을 확인하였다.

`git pull` 은 원격 저장소의 변경 내용을 가져온 뒤 현재 로컬 브랜치에 반영하는 명령어이다.

사용한 명령어

```
git pull
```

수행 결과

원격 저장소와 로컬 저장소가 이미 동일한 상태여서 다음 메시지가 출력되었다.

```
Already up to date.
```

이를 통해 로컬 저장소가 원격 저장소의 최신 상태와 동기화되어 있음을 확인하였다.

 스크린샷 첨부

 26_git_pull.png

```
PS C:\Users\jmkmo\Desktop\Hello-World> git pull
Already up to date.
```

7. 프로젝트 설계

프로그램은 사용자가 메뉴를 선택하여 원하는 기능을 실행하는 콘솔(Console) 프로그램으로 설계하였다.

프로그램이 실행되면 반복적으로 메뉴를 출력하고, 사용자가 번호를 입력하면 해당 기능을 수행하도록 구현하였다.

메뉴는 다음과 같이 구성하였다.

- - 1. 프롬프트 추가
- - 1. 프롬프트 목록
- - 1. 카테고리별 조회
- - 1. 프롬프트 검색
-

1. 프롬프트 상세 보기

-

1. 즐겨찾기 관리

-

1. 즐겨찾기 목록

-

1. 종료

사용자가 0을 입력하기 전까지 프로그램이 계속 실행되도록 설계하였다.

 **스크린샷 첨부**

 **06_menu_function.png**

```
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> python prompt_manager.py
=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 1
아직 준비 중인 기능입니다.

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 9
잘못된 번호입니다. 다시 입력해주세요.

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 0
프로그램을 종료합니다.
```

8. 데이터 구조 설계

프로젝트에서는 여러 개의 프롬프트를 관리하기 위해 **리스트(List)** 안에 **딕셔너리(Dictionary)**를 저장하는 구조를 사용하였다.

예시는 다음과 같다.

```
prompts= [
    {"title": "회의록 요약", "content": "...", "category": "텍스트 생성", "favorite": True
    }
]
```

리스트(List)를 사용한 이유

프로그램에서는 여러 개의 프롬프트를 저장해야 한다.

리스트를 사용하면 프롬프트를 순서대로 저장할 수 있으며 반복문을 이용하여 모든 데이터를 쉽게 출력할 수 있다.

또한 새로운 프롬프트를 추가할 때 `append()`를 이용하여 간단하게 데이터를 추가할 수 있다는 장점이 있다.

딕셔너리(Dictionary)를 사용한 이유

프롬프트 하나에는

- 제목
- 내용
- 카테고리
- 즐겨찾기 여부

와 같이 여러 개의 정보가 존재한다.

이러한 데이터를 하나의 객체처럼 관리하기 위해 Dictionary를 사용하였다.

Dictionary를 사용하면

```
prompt["title"]
```

처럼 원하는 데이터에 이름으로 접근할 수 있어 코드의 가독성이 좋아지고 관리하기 쉬워진다.

 **스크린샷 첨부**

 **08_prompt_list.png**

```

PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> python prompt_manager.py

=== 너만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. 종료
선택: 2

=== 프롬프트 목록 ===
1. [텍스트 생성] 회의록 요약 ★
2. [자동화] AI 뉴스 요약
3. [영상 생성] 광고 영상 생성 ★

총 3개의 프롬프트

```

9. 기능 구현

9-1. 프롬프트 추가

기능 설명

프롬프트 추가 기능은 사용자가 새로운 프롬프트를 프로그램에 등록할 수 있도록 구현한 기능이다.

사용자는 메뉴에서 **1번**을 선택한 후 제목, 내용, 카테고리를 입력하여 새로운 프롬프트를 등록할 수 있다.

등록된 프롬프트는 프로그램 실행 중 리스트에 저장되며, 즐겨찾기 기본값은 **False**로 설정하였다.

구현 과정

사용자가 입력한 값을 변수에 저장한 후 Dictionary 형태로 생성하였다.

생성된 Dictionary는 prompts 리스트에 append()를 사용하여 추가하였다.

입력값이 비어 있는 경우에는 while 반복문을 이용하여 다시 입력하도록 구현하였다.

이를 통해 빈 데이터가 저장되지 않도록 입력을 검증하였다.

구현 이유

입력 검증을 추가한 이유는 제목이나 내용이 없는 프롬프트가 저장되는 것을 방지하기 위해서이다.

또한 Dictionary를 사용하여 하나의 프롬프트 정보를 하나의 객체처럼 관리할 수 있도록 설계하였다.

실행 결과

사용자가

- 제목
- 내용
- 카테고리

를 입력하면 새로운 프롬프트가 추가된다.

등록이 완료되면

프롬프트가 추가되었습니다.

메시지를 출력하도록 구현하였다.

 **스크린샷 첨부**

 **11_add_prompt_success.png**

```

PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> python prompt_manager.py

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 1

=== 프롬프트 추가 ===
제목: 테스트 프롬프트
내용: 안녕하세요

카테고리 선택:
1) 텍스트 생성
2) 이미지 생성
3) 영상 생성
4) 페르소나
5) 자동화
6) 기타
선택: 1
프롬프트가 추가되었습니다!

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 2

=== 프롬프트 목록 ===
1. [텍스트 생성] 회의록 요약 ★
2. [자동화] AI 뉴스 요약
3. [영상 생성] 광고 영상 생성 ★
4. [텍스트 생성] 테스트 프롬프트

총 4개의 프롬프트

```

9-2. 프롬프트 목록

기능 설명

프롬프트 목록 기능은 프로그램에 저장된 모든 프롬프트를 출력하는 기능이다.

메뉴에서 **2번**을 선택하면 저장된 프롬프트를 확인할 수 있다.

출력 항목

- 번호

- 제목
- 카테고리
- 즐겨찾기 여부

즐거찾기로 등록된 프롬프트는 ★ 기호를 함께 출력하도록 구현하였다.

구현 과정

for 반복문과 enumerate()를 이용하여 번호를 자동으로 출력하였다.

프롬프트의 favorite 값이 True이면 ★ 기호를 출력하고 False이면 아무것도 출력하지 않도록 구현하였다.

등록된 프롬프트가 없는 경우에는 안내 문구를 출력하도록 예외 처리를 수행하였다.

구현 이유

enumerate()를 사용하면 번호를 직접 증가시키지 않아도 자동으로 번호를 출력할 수 있기 때문에 코드가 간결해진다.

또한 즐겨찾기 여부를 한눈에 확인할 수 있도록 ★ 기호를 사용하였다.

실행 결과

등록된 모든 프롬프트를 번호와 함께 출력한다.

출력 정보

- 제목
- 카테고리
- 즐겨찾기

총 프롬프트 개수도 함께 출력한다.

 **스크린샷 첨부**

 **08_prompt_list.png**

```

PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> python prompt_manager.py

=== 너만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. 종료
선택: 2

=== 프롬프트 목록 ===
1. [텍스트 생성] 회의록 요약 ★
2. [자동화] AI 뉴스 요약
3. [영상 생성] 광고 영상 생성 ★

총 3개의 프롬프트

```

9-3. 카테고리별 조회

기능 설명

카테고리를 선택하면 해당 카테고리에 속한 프롬프트만 출력하는 기능이다.
메뉴에서 **3번**을 선택하여 사용할 수 있다.

구현 과정

카테고리 목록을 출력한 후 사용자가 번호를 입력하도록 구현하였다.
선택된 카테고리와 `prompt["category"]` 값을 비교하여 일치하는 프롬프트만 출력하였다.
일치하는 프롬프트가 하나도 없는 경우에는

해당 카테고리의 프롬프트가 없습니다.

메시지를 출력하도록 구현하였다.

구현 이유

for 반복문을 이용하여 모든 프롬프트를 검사한 후 조건문을 통해 원하는 데이터만 출력하도록 구현하였다.

필터(Filter) 기능과 비슷한 방식으로 동작한다.

실행 결과

선택한 카테고리의 프롬프트만 출력된다.

총 프롬프트 개수도 함께 출력된다.

스크린샷 첨부

 14_category_filter.png

```
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> python prompt_manager.py
=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 3

=== 카테고리별 조회 ===
1. 텍스트 생성
2. 이미지 생성
3. 영상 생성
4. 페르소나
5. 자동화
6. 기타
선택: 1

[텍스트 생성] 카테고리 프롬프트
1. 회의록 요약 ★

총 1개의 프롬프트

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 3

=== 카테고리별 조회 ===
1. 텍스트 생성
2. 이미지 생성
3. 영상 생성
4. 페르소나
5. 자동화
6. 기타
선택: 2

[이미지 생성] 카테고리 프롬프트
해당 카테고리의 프롬프트가 없습니다.
```

9-4. 프롬프트 검색

기능 설명

검색어를 입력하면 제목 또는 내용에 검색어가 포함된 프롬프트를 검색하는 기능이다.

메뉴에서 **4번**을 선택하여 사용할 수 있다.

구현 과정

사용자가 입력한 검색어를 변수에 저장하였다.

for 반복문을 이용하여 모든 프롬프트를 검사하였다.

조건문에서는

- 제목(title)

- 내용(content)

두 곳 모두 검색하도록 구현하였다.

검색 결과는 새로운 리스트(results)에 저장한 후 출력하였다.

검색 결과가 없는 경우에는

검색 결과가 없습니다.

메시지를 출력하도록 구현하였다.

구현 이유

검색 결과를 별도의 리스트에 저장하면 필요한 데이터만 출력할 수 있고, 검색 결과 개수도 쉽게 계산할 수 있다.

또한 제목뿐 아니라 내용까지 함께 검색하여 사용성을 높였다.

실행 결과

검색어가 포함된 프롬프트만 출력된다.

검색된 프롬프트 개수도 함께 출력된다.

 스크린샷 첨부

 16_search_prompt.png

```

PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> python prompt_manager.py

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 4

=== 프롬프트 검색 ===
검색어: 회의

검색 결과:
1. [텍스트 생성] 회의록 요약 ★

1개의 프롬프트를 찾았습니다.

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 4

=== 프롬프트 검색 ===
검색어: 뉴스

검색 결과:
1. [자동화] AI 뉴스 요약

1개의 프롬프트를 찾았습니다.

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 4

=== 프롬프트 검색 ===
검색어: 과일
검색 결과가 없습니다.

```

9-5. 프롬프트 상세 보기

기능 설명

프롬프트 번호를 입력하면 해당 프롬프트의 상세 정보를 출력하는 기능이다.

메뉴에서 **5번**을 선택하여 사용할 수 있다.

출력 항목

- 제목
 - 카테고리
 - 즐겨찾기 여부
 - 내용 전체
-

구현 과정

사용자가 입력한 번호를 이용하여 리스트의 해당 위치를 찾도록 구현하였다.

번호가 범위를 벗어난 경우에는

올바른 프롬프트 번호를 입력해주세요.

메시지를 출력하고 다시 입력받도록 구현하였다.

구현 이유

사용자가 잘못된 번호를 입력할 가능성을 고려하여 입력 검증을 추가하였다.

이를 통해 프로그램 오류를 방지하였다.

실행 결과

선택한 프롬프트의 상세 정보가 출력된다.

 **스크린샷 첨부**

 **18_prompt_detail.png**

```

PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> python prompt_manager.py

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 5

=== 프롬프트 상세 보기 ===
프롬프트 번호 입력: 1

제목: 회의록 요약
카테고리: 텍스트 생성
즐거찾기: ★

내용:
회의 내용을 핵심 요약, 결정사항, 담당자, 일정으로 정리해 주세요.

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 5

=== 프롬프트 상세 보기 ===
프롬프트 번호 입력: 99
올바른 프롬프트 번호를 입력해주세요.

```

9-6. 즐겨찾기 관리

기능 설명

프롬프트 번호를 입력하여 즐겨찾기를 추가하거나 해제하는 기능이다.

메뉴에서 **6번**을 선택하여 사용할 수 있다.

구현 과정

선택한 프롬프트의 favorite 값을

True ↔ False

로 변경하도록 구현하였다.

이미 즐겨찾기인 경우에는 해제하고,

즐거찾기가 아닌 경우에는 추가하도록 구현하였다.

구현 이유

Boolean(True / False)을 사용하면 즐겨찾기 상태를 간단하게 관리할 수 있다.

복잡한 조건 없이

True → False

False → True

형태로 변경할 수 있다는 장점이 있다.

실행 결과

즐거찾기 추가 및 해제 메시지를 출력한다.

 **스크린샷 첨부**

 **20_favorite_toggle.png**

```
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> python .\prompt_manager.py

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 6

=== 즐겨찾기 관리 ===
프롬프트 번호 입력: 2
'AI 뉴스 요약' 프롬프트를 즐겨찾기에 추가했습니다!

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 6

=== 즐겨찾기 관리 ===
프롬프트 번호 입력: 2
'AI 뉴스 요약' 프롬프트의 즐겨찾기를 해제했습니다!
```

9-7. 즐겨찾기 목록

기능 설명

즐거찾기로 등록된 프롬프트만 출력하는 기능이다.

메뉴에서 **7번**을 선택하여 사용할 수 있다.

구현 과정

favorite 값이 True인 프롬프트만 새로운 리스트에 저장하였다.

저장된 리스트를 반복문으로 출력하였다.

즐거찾기가 하나도 없는 경우에는 안내 문구를 출력하도록 구현하였다.

구현 이유

전체 프롬프트를 모두 출력하는 것이 아니라 필요한 데이터만 보여주기 위해 별도의 리스트를 생성하였다.

이를 통해 사용자가 자주 사용하는 프롬프트만 빠르게 확인할 수 있도록 하였다.

실행 결과

즐거찾기로 등록된 프롬프트만 출력된다.

총 즐겨찾기 개수도 함께 출력된다.

 **스크린샷 첨부**

 **22_favorite_list.png**

```

PS C:\Users\jmkmo\Desktop\Prompt Manager Project> python prompt_manager.py

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 6

=== 즐겨찾기 관리 ===
프롬프트 번호 입력: 2
'AI 뉴스 요약' 프롬프트를 즐겨찾기에 추가했습니다!

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 7

=== 즐겨찾기 목록 ===
1. [텍스트 생성] 회의록 요약 ★
2. [자동화] AI 뉴스 요약 ★
3. [영상 생성] 광고 영상 생성 ★

총 3개의 즐겨찾기

```

10. Git을 이용한 버전 관리

Git을 사용한 이유

프로젝트를 개발하면서 기능을 하나씩 추가하였다.

기능을 추가할 때마다 변경 사항을 저장하고 이전 버전으로 되돌릴 수 있도록 Git을 이용하여 버전 관리를 수행하였다.

또한 GitHub를 원격 저장소로 사용하여 프로젝트를 백업하고 관리하였다.

Git을 사용함으로써 기능별 변경 이력을 확인할 수 있었으며, 새로운 기능을 개발하는 과정도 체계적으로 관리할 수 있었다.

Git 작업 과정

프로젝트를 생성한 후 Git 저장소를 초기화하였다.

```
git init
```

프로젝트 파일을 Git이 관리하도록 추가하였다.

```
git add .
```

기능을 하나씩 구현할 때마다 Commit을 수행하였다.

```
git commit -m "커밋 메시지"
```

GitHub 원격 저장소에 프로젝트를 업로드하였다.

```
git push
```

 스크린샷 첨부

 24_git_log_graph.png

```
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git log --oneline --graph
* c2681a5 (HEAD -> main, origin/main) feat: add favorite list
* afec36e feat: add favorite management
* b4e8cd9 feat: add prompt detail
* 2775052 feat: add serach feature
* 533b27d feat: add category filter
* 2602f2d (origin/feature/prompt-list, feature/prompt-list) feat: add prompt
t
* 288382d feat: add main menu
:...skipping...
* c2681a5 (HEAD -> main, origin/main) feat: add favorite list
* afec36e feat: add favorite management
* b4e8cd9 feat: add prompt detail
* 2775052 feat: add serach feature
* 533b27d feat: add category filter
* 2602f2d (origin/feature/prompt-list, feature/prompt-list) feat: add prompt
t
* 288382d feat: add main menu
* 34ef6d2 feat: add main menu
* 48bda80 chore: initialize project structure
```

11. Branch 관리

Branch를 사용한 이유

기능을 개발할 때 main 브랜치에서 바로 작업하는 대신 feature 브랜치를 생성하여 개발을 진행하였다.

이를 통해 안정적인 main 브랜치를 유지하면서 새로운 기능을 안전하게 개발할 수 있었다.

기능 개발이 완료된 후에는 Merge를 수행하여 main 브랜치에 반영하였다.

실제 협업 환경에서도 동일한 방식으로 Branch를 이용하여 개발이 이루어진다.

수행 과정

feature 브랜치를 생성하였다.

```
git checkout -b feature/prompt-list
```

feature 브랜치에서 기능을 구현하였다.

기능 구현이 완료된 후 main 브랜치로 이동하였다.

```
git checkout main
```

Merge를 수행하였다.

```
git merge feature/prompt-list
```

 **스크린샷 첨부**

 **10_branch_create.png**

```
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git checkout -b feature/prompt-list
Switched to a new branch 'feature/prompt-list'
```

 **스크린샷 첨부**

 **13_branch_merge_push.png**

```

PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git merge feature/prompt-
list
Updating 288382d..2602f2d
Fast-forward
 prompt_manager.py | 63 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
+++
1 file changed, 62 insertions(+), 1 deletion(-)
PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git push
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/jmkmo520-glitch/Prompt_Manager_Project.git
288382d..2602f2d main -> main

```

12. Commit 관리

기능을 하나 구현할 때마다 Commit을 수행하였다.

Commit을 기능 단위로 수행함으로써 변경 사항을 쉽게 확인할 수 있도록 관리하였다.

프로젝트에서 수행한 주요 Commit은 다음과 같다.

Commit	설명
chore: initialize project structure	프로젝트 초기 생성
feat: add main menu	메인 메뉴 구현
feat: add prompt	프롬프트 추가 기능 구현
feat: add category filter	카테고리 조회 기능 구현
feat: add search feature	검색 기능 구현
feat: add prompt detail	상세 보기 기능 구현
feat: add favorite management	즐거찾기 관리 기능 구현
feat: add favorite list	즐거찾기 목록 구현
docs: update README	README 작성
docs: add screenshots	스크린샷 추가

기능별 Commit을 수행하여 프로젝트 변경 이력을 체계적으로 관리하였다.

 **스크린샷 첨부**

 **23_git_log_graph.png**

```

PS C:\Users\jmkmo\Desktop\Prompt_Manager_Project> git log --oneline --graph
* c2681a5 (HEAD -> main, origin/main) feat: add favorite list
* afec36e feat: add favorite management
* b4e8cd9 feat: add prompt detail
* 2775052 feat: add search feature
* 533b27d feat: add category filter
* 2602f2d (origin/feature/prompt-list, feature/prompt-list) feat: add prompt
t
* 288382d feat: add main menu
:...skipping...
* c2681a5 (HEAD -> main, origin/main) feat: add favorite list
* afec36e feat: add favorite management
* b4e8cd9 feat: add prompt detail
* 2775052 feat: add search feature
* 533b27d feat: add category filter
* 2602f2d (origin/feature/prompt-list, feature/prompt-list) feat: add prompt
t
* 288382d feat: add main menu
* 34ef6d2 feat: add main menu
* 48bda80 chore: initialize project structure

```

13. Python에서 사용한 주요 문법

프로젝트에서는 Python의 다양한 기초 문법을 활용하였다.

문법	사용 목적
변수	데이터 저장
리스트(List)	여러 개의 프롬프트 저장
딕셔너리(Dictionary)	하나의 프롬프트 정보 저장
함수(Function)	기능별 코드 분리
조건문(if)	메뉴 선택 및 조건 처리
반복문(for)	프롬프트 출력
반복문(while)	프로그램 반복 실행
enumerate()	번호 자동 출력
input()	사용자 입력
print()	화면 출력
strip()	공백 제거
lower()	대소문자 구분 없이 검색
Boolean(True / False)	즐거찾기 관리

14. 주요 함수 설명

프로그램은 기능별로 함수를 분리하여 구현하였다.

함수	역할
show_menu()	메뉴 출력
add_prompt()	새로운 프롬프트 등록
show_prompt_list()	전체 프롬프트 출력
show_category()	카테고리별 조회
search_prompt()	제목 및 내용 검색
show_prompt_detail()	프롬프트 상세 정보 출력
manage_favorite()	즐거찾기 추가 및 해제
show_favorites()	즐거찾기 목록 출력
main()	프로그램 실행 및 메뉴 제어

함수를 기능별로 분리하여 작성함으로써 코드의 가독성을 높였으며, 유지보수가 쉽도록 구현하였다.

15. 프로그램 동작 과정

프로그램은 다음과 같은 순서로 동작하도록 설계하였다.

프로그램 시작

↓

메뉴 출력

↓

사용자 입력

↓

입력값 확인

↓

선택한 기능 실행

↓

메뉴 다시 출력

↓

0 입력

↓

프로그램 종료

프로그램 종료 전까지 while 반복문을 이용하여 메뉴를 계속 출력하도록 구현하였다.

16. 구현 과정에서 고려한 사항

프로그램을 구현하면서 다음 사항을 고려하였다.

입력 검증

빈 문자열이나 잘못된 번호를 입력하는 경우 프로그램이 종료되지 않고 다시 입력받도록 구현하였다.

이를 통해 사용자 입력 오류를 방지하였다.

함수 분리

기능마다 별도의 함수를 작성하여 코드의 가독성과 유지보수성을 높였다.

각 함수는 하나의 기능만 담당하도록 구현하였다.

데이터 관리

프롬프트 정보를 리스트와 딕셔너리를 이용하여 구조적으로 관리하였다.

이를 통해 검색, 조회, 추가 등의 기능을 쉽게 구현할 수 있었다.

사용자 편의성

즐거찾기에는 ★ 기호를 출력하여 사용자가 자주 사용하는 프롬프트를 쉽게 확인할 수 있도록 구현하였다.

또한 검색 결과와 즐거찾기 개수를 함께 출력하여 사용성을 높였다.

17. 프로젝트 결과

본 프로젝트를 통해 생성형 AI 프롬프트를 효율적으로 관리할 수 있는 Python 콘솔 프로그램을 구현하였다.

프로그램은 사용자가 메뉴를 선택하여 원하는 기능을 실행할 수 있도록 설계하였으며, 프롬프트를 추가하고 목록을 조회하며, 카테고리별 조회와 검색, 상세 보기, 즐겨찾기 관리 등의 기능을 제공하도록 구현하였다.

또한 Git을 이용하여 기능별 Commit을 수행하였으며, Branch 생성 및 Merge를 통해 실제 개발 프로젝트와 유사한 방식으로 버전 관리를 수행하였다.

GitHub를 이용하여 프로젝트를 원격 저장소에 업로드하고 README와 프로젝트 진행 과정의 스크린샷을 함께 관리하였다.

구현 완료 기능

기능	구현 여부
메인 메뉴	완료
프롬프트 추가	완료
프롬프트 목록	완료
카테고리별 조회	완료
프롬프트 검색	완료
프롬프트 상세 보기	완료
즐거찾기 관리	완료
즐거찾기 목록	완료
프로그램 종료	완료

 **스크린샷 첨부**

 **22_favorite_list.png**

```

PS C:\Users\jmkmo\Desktop\Prompt Manager Project> python prompt_manager.py

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 6

=== 즐겨찾기 관리 ===
프롬프트 번호 입력: 2
'AI 뉴스 요약' 프롬프트를 즐겨찾기에 추가했습니다!

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
0. 종료
선택: 7

=== 즐겨찾기 목록 ===
1. [텍스트 생성] 회의록 요약 ★
2. [자동화] AI 뉴스 요약 ★
3. [영상 생성] 광고 영상 생성 ★

총 3개의 즐겨찾기

```

18. 프로젝트를 통해 학습한 내용

이번 프로젝트를 수행하면서 Python뿐만 아니라 Git과 GitHub를 함께 학습할 수 있었다.

프로젝트를 진행하기 전에는 Python 문법과 Git 사용 방법이 익숙하지 않았지만, 기능을 하나씩 구현하고 Git으로 변경 사항을 관리하는 과정을 반복하면서 프로그램 개발의 전체 흐름을 이해할 수 있었다.

Python

이번 프로젝트를 통해 다음 내용을 학습하였다.

- 변수 선언 및 활용
- 문자열(String) 처리
- 리스트(List)를 이용한 데이터 관리
- 딕셔너리(Dictionary)를 이용한 데이터 구조 설계

- 함수(Function)를 이용한 기능 분리
 - 조건문(if, elif, else)
 - 반복문(for, while)
 - enumerate() 활용
 - Boolean(True / False)
 - 사용자 입력 처리(input)
 - 입력값 검증
 - 검색 기능 구현
-

Git

Git을 이용하여 프로젝트를 버전 관리하는 방법을 학습하였다.

실습을 통해 다음 명령어를 직접 사용하였다.

- git init
- git add
- git commit
- git push
- git checkout
- git merge
- git status
- git log

기능별 Commit을 수행하면서 변경 사항을 관리하는 방법을 이해할 수 있었다.

GitHub

GitHub를 이용하여 원격 저장소를 생성하고 프로젝트를 업로드하였다.

또한 README를 작성하여 프로젝트를 설명하고 screenshots 폴더를 이용하여 프로젝트 진행 과정을 함께 관리하였다.

19. 프로젝트 수행 소감

이번 프로젝트를 수행하면서 Python의 기본 문법만으로도 다양한 기능을 구현할 수 있다는 것을 경험하였다.

처음에는 함수와 반복문, 리스트와 딕셔너리의 사용 방법이 익숙하지 않았지만 기능을 하나씩 구현하면서 점차 이해할 수 있었다.

또한 Git과 GitHub를 이용하여 프로젝트를 관리하는 과정에서 기능별 Commit과 Branch 관리의 중요성을 배울 수 있었다.

이번 프로젝트를 통해 프로그램을 작성하는 것뿐만 아니라 변경 사항을 체계적으로 관리하는 개발자의 작업 방식도 함께 경험할 수 있었다.

20. 향후 개선 사항

현재 프로그램은 콘솔 환경에서 실행되는 프로그램이며 프로그램이 종료되면 추가한 데이터가 저장되지 않는다.

향후에는 다음과 같은 기능을 추가하여 프로그램을 확장할 계획이다.

데이터 저장 기능

현재는 프로그램 실행 중에만 데이터가 유지된다.

JSON 파일 저장 및 불러오기 기능을 추가하여 프로그램을 종료한 후에도 데이터를 유지할 수 있도록 개선할 계획이다.

프롬프트 수정 기능

등록된 프롬프트를 수정할 수 있는 기능을 추가할 예정이다.

프롬프트 삭제 기능

불필요한 프롬프트를 삭제할 수 있는 기능을 추가할 예정이다.

Markdown 내보내기

등록한 프롬프트를 Markdown 파일로 저장하여 노션 등 다른 프로그램에서도 활용할 수 있도록 개선할 예정이다.

조회수 기능

자주 사용하는 프롬프트를 확인할 수 있도록 조회수 기능을 추가할 예정이다.

정렬 기능

조회수 순, 제목 순, 카테고리 순 등 다양한 기준으로 정렬할 수 있도록 개선할 예정이다.

21. GitHub Repository

프로젝트는 GitHub를 이용하여 관리하였다.

GitHub 저장소에는 다음 파일을 업로드하였다.

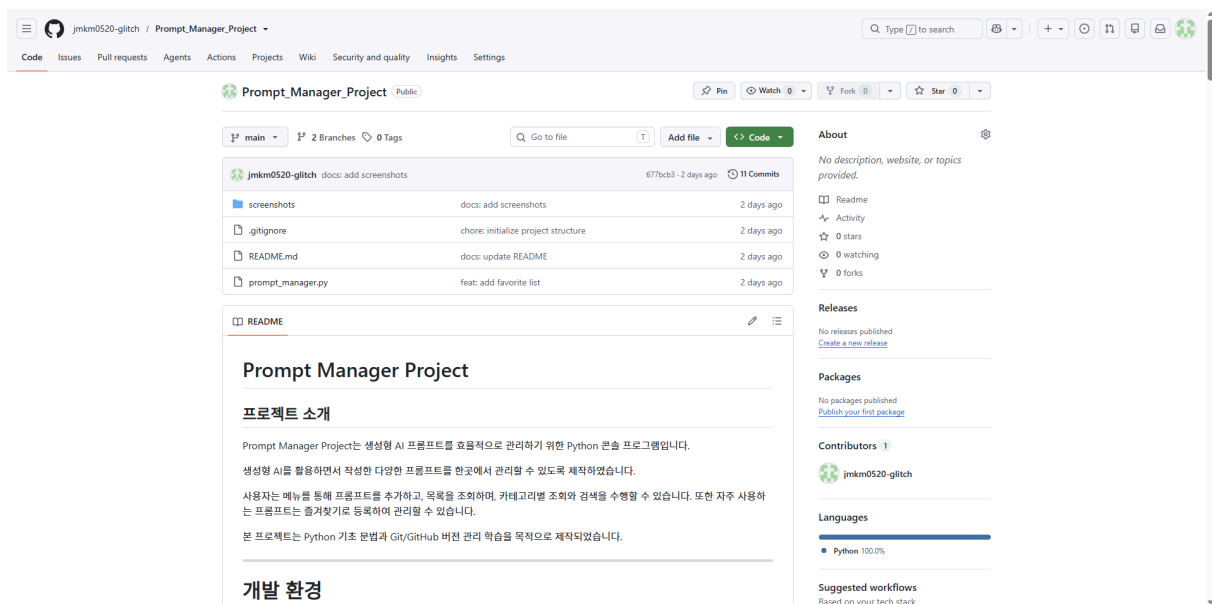
- prompt_manager.py
- README.md
- .gitignore
- screenshots 폴더

GitHub 저장소를 통해 프로젝트의 소스코드와 변경 이력을 함께 확인할 수 있다.

 스크린샷 첨부

 GitHub 메인 화면 캡처

(README와 screenshots 폴더가 보이는 화면)



22. 결론

이번 프로젝트에서는 Python을 이용하여 생성형 AI 프롬프트를 관리할 수 있는 콘솔 프로그램을 구현하였다.

프로그램 개발 과정에서 Python의 기본 문법을 활용하여 다양한 기능을 구현하였으며, Git과 GitHub를 이용하여 프로젝트를 체계적으로 관리하였다.

기능별 Commit과 Branch 생성 및 Merge를 직접 수행하면서 실제 개발 프로젝트의 버전 관리 과정을 경험할 수 있었으며, README와 프로젝트 문서를 작성하여 프로젝트를 체계적으로 정리하였다.

이번 프로젝트를 통해 Python 프로그래밍 기초뿐만 아니라 Git을 활용한 버전 관리와 GitHub를 이용한 프로젝트 관리 방법까지 함께 학습할 수 있었다.

앞으로는 JSON 저장 기능과 프롬프트 수정 및 삭제 기능 등을 추가하여 더욱 완성도 높은 프로젝트로 발전시킬 계획이다.